3. 프로토콜 레이어 동작 분석

3.1 프로토콜 개요

3.1.1 기본 특성

iroh-blob 프로토콜은 **QUIC 기반의 요청-응답 프로토콜**로, 다음과 같은 특징을 가집니다:

- **ALPN**: `/iroh-bytes/4` (Application Layer Protocol Negotiation)
- **전송 계층**: QUIC (UDP 기반, 다중 스트림 지원)
- **메시지 형식**: postcard 직렬화 (바이너리)
- **최대 메시지 크기**: 100MiB

```
/// The ALPN used with quic for the iroh blobs protocol.
pub const ALPN: &[u8] = b"/iroh-bytes/4";
```

```
/// Maximum message size is limited to 100MiB for now.
pub const MAX_MESSAGE_SIZE: usize = 1024 * 1024;
```

3.1.2 프로토콜 설계 목표

1. **데이터 무결성 우선**: 성능보다 무결성을 중시
2. **크기 제한 없음**: 테라바이트급 파일 지원
3. **효율적인 범위 요청**: 16KiB 단위 세밀한 제어
4. **다중 작은 파일 효율성**: 라운드트립 최소화

3.2 요청 메시지 구조

3.2.1 Request 열거형

```
#[derive(Deserialize, Serialize, Debug, PartialEq, Eq, Clone, From)]
pub  enum Request {
Get(GetRequest), // 데이터 요청
Observe(ObserveRequest), // 상태 관찰
Slot2, // 미래 확장용
Slot3,
Slot4,
Slot5,
Slot6,
Slot7,
Push(PushRequest), // 데이터 업로드
GetMany(GetManyRequest), // 다중 블롭 요청
}
```

3.2.2 요청 타입 식별

```
#[derive(Deserialize, Serialize, Debug, PartialEq, Eq, Clone, Copy, MaxSize)]
pub  enum RequestType {
Get,
Observe,
```

```
Slot2, Slot3, Slot4, Slot5, Slot6, Slot7,
Push,
GetMany,
}
```

효율적인 파싱을 위해 요청 타입을 먼저 읽고, 해당하는 구조체를 역직렬화합니다.

3.3 GetRequest - 기본 데이터 요청

3.3.1 구조 정의

```rust
#[derive(Deserialize, Serialize, Debug, PartialEq, Eq, Clone, Hash)]
pub struct GetRequest {
pub hash: Hash, // 요청할 블롭의 해시
pub ranges: ChunkRangesSeq, // 요청할 청크 범위들
}
```

3.3.2 요청 생성 방법

전체 블롭 요청

```rust
let request = GetRequest::blob(hash);
// 내부적으로: ChunkRangesSeq::from_ranges([ChunkRanges::all()])
```

범위 지정 요청

```
let request = GetRequest::blob_ranges(hash, ChunkRanges::bytes(0..1024));
```

컬렉션 전체 요청

```
let request = GetRequest::all(hash);
// 내부적으로: ChunkRangesSeq::all() (무한 시퀀스)
```

3.3.3 빌더 패턴

```
let request = GetRequest::builder()
.root(ChunkRanges::all()) // 루트 블롭 전체
.child(1, ChunkRanges::chunks(1000000..)) // 두 번째 자식의 일부
.next(ChunkRanges::all()) // 나머지 자식들 전체
.build_open(hash);
```

3.4 ChunkRangesSeq - 범위 시퀀스

3.4.1 설계 목적

ChunkRangesSeq는 다음과 같은 복잡한 요청 패턴을 효율적으로 표현하기 위해 설계되었습니다:

1. **단일 블롭의 특정 범위들**
2. **컬렉션의 특정 자식들**
3. **각 자식의 특정 범위들**
4. **무한 시퀀스** (크기를 모르는 컬렉션)

3.4.2 와이어 형식 최적화

```
// 내부 표현: (offset, ChunkRanges) 쌍들의 시퀀스
// 와이어 형식: 런 길이 인코딩 + 차분 인코딩
pub struct ChunkRangesSeq(SmallVec<[(u64, ChunkRanges); 1]>);
```

최적화 기법:

- **런 길이 인코딩**: 반복되는 범위 압축
- **차분 인코딩**: 연속된 오프셋의 차이만 저장
- **가변 길이 정수**: postcard의 varint 활용

3.4.3 사용 예시

단일 블롭 부분 요청

```
// 0-10 청크와 100-110 청크 요청
let ranges = ChunkRanges::chunks(0..10) | ChunkRanges::chunks(100..110);
let request = GetRequest::blob_ranges(hash, ranges);
```

컬렉션의 특정 자식들

```
let request = GetRequest::builder()
.child(0, ChunkRanges::all()) // 첫 번째 자식 전체
.child(2, ChunkRanges::all()) // 세 번째 자식 전체
.build(hash);
```

각 자식의 첫 청크만

```
let request = GetRequest::builder()
.root(ChunkRanges::all()) // 컬렉션 자체
.next(ChunkRanges::chunk(0)) // 각 자식의 첫 청크
.build_open(hash); // 무한 반복
```

3.5 GetManyRequest - 다중 블롭 요청

3.5.1 구조 및 용도

```
#[derive(Deserialize, Serialize, Debug, PartialEq, Eq, Clone)]
pub struct GetManyRequest {
pub hashes: Vec<Hash>, // 요청할 블롭들의 해시
pub ranges: ChunkRangesSeq, // 각 블롭의 범위
}
```

GetManyRequest는 **관련 없는 여러 블롭을 한 번에 요청**할 때 사용합니다. HashSeq와 달리 서버에 컬렉션이 없어도 됩니다.

3.5.2 자동 중복 제거

```
impl<I: Into<Hash>> FromIterator<I> for GetManyRequest {
fn from_iter<T: IntoIterator<Item = I>>(iter: T) -> Self {
let mut res = iter.into_iter().map(Into::into).collect::<Vec<Hash>>();
```

```
res.sort(); // 정렬

res.dedup(); // 중복 제거

let  n = res.len() as u64;

Self {

hashes: res,

ranges: ChunkRangesSeq(smallvec::smallvec![

(0, ChunkRanges::all()),

(n, ChunkRanges::empty())

]),

}

}

}
```

3.5.3 사용 시나리오

```
// 여러 작은 파일들을 한 번에 요청

let  request = GetManyRequest::builder()

.hash(config_hash, ChunkRanges::all())

.hash(readme_hash, ChunkRanges::all())

.hash(license_hash, ChunkRanges::all())

.build();
```

3.6 PushRequest - 데이터 업로드

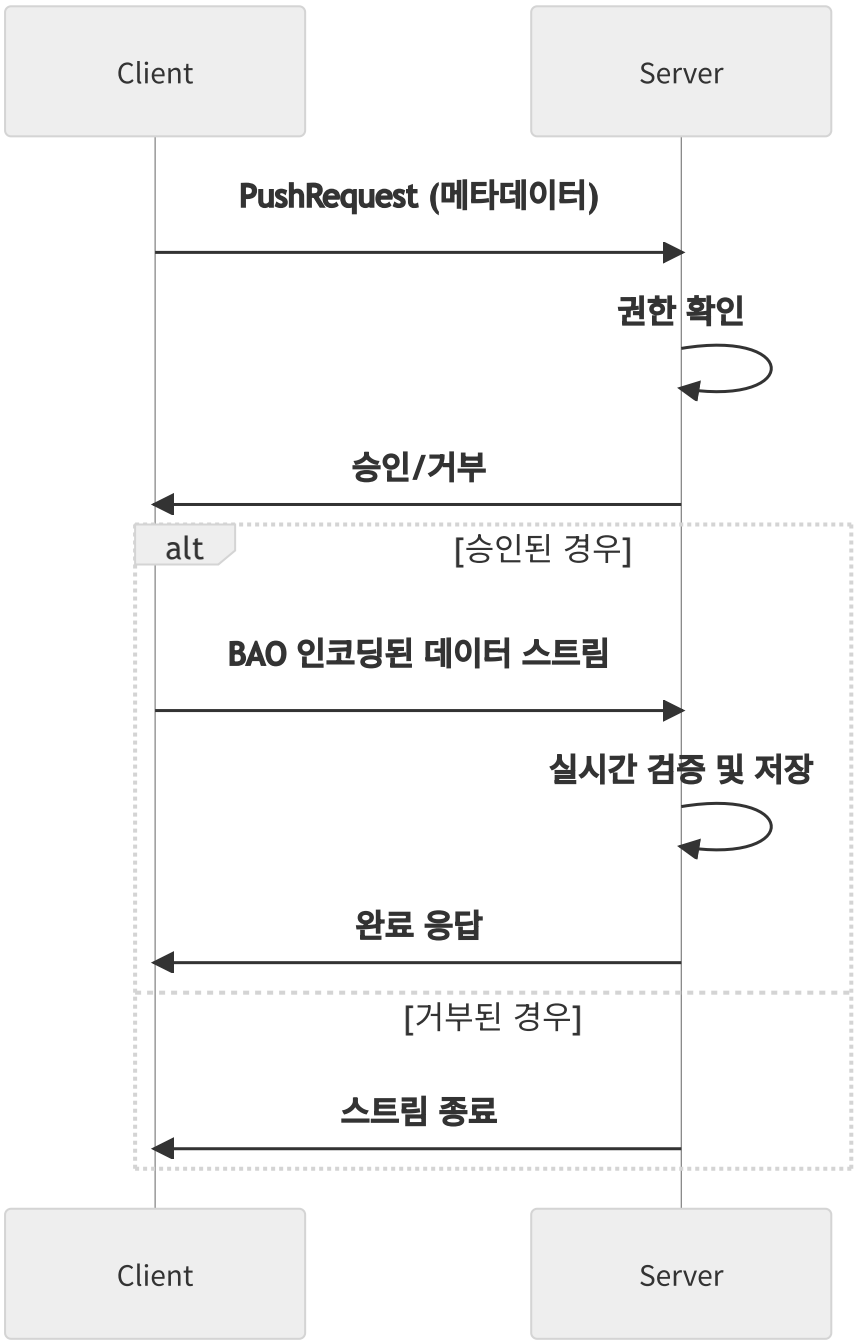
3.6.1 구조 정의

```rust
#[derive(Deserialize, Serialize, Debug, PartialEq, Eq, Clone)]

pub struct PushRequest(GetRequest);
```

PushRequest는 GetRequest를 래핑하여 **업로드할 데이터의 구조를 설명**합니다.

3.6.2 Push 프로토콜 흐름



3.6.3 보안 고려사항

Push 요청은 **명시적 승인**이 필요합니다:

```
// Provider에서 Push 요청 승인 확인
let permitted = context.authorize_push_request(&hash, &ranges).await;
if !permitted {
debug!("push request not authorized");
return Ok(());
}
```

3.7 ObserveRequest - 실시간 상태 관찰

3.7.1 구조 및 용도

```
#[derive(Deserialize, Serialize, Debug, PartialEq, Eq, Clone, Hash)]
pub struct ObserveRequest {
pub hash: Hash, // 관찰할 블롭
pub ranges: RangeSpec, // 관찰할 범위
}
```

ObserveRequest는 **블롭의 다운로드 진행 상황을 실시간으로 관찰**하는 데 사용됩니다.

3.7.2 응답 형식

```
#[derive(Deserialize, Serialize, Debug, PartialEq, Eq)]
```

```rust
pub struct ObserveItem {
pub size: u64, // 블롭 크기
pub ranges: ChunkRanges, // 사용 가능한 청크들
}
```

3.7.3 스트리밍 업데이트

```rust
// 초기 상태 전송
send_observe_item(writer, &initial_bitfield).await?;

// 변경사항만 전송
loop {
let new_bitfield = stream.next().await?;
let diff = old_bitfield.diff(&new_bitfield);
if !diff.is_empty() {
send_observe_item(writer, &diff).await?;
}
old_bitfield = new_bitfield;
}
```

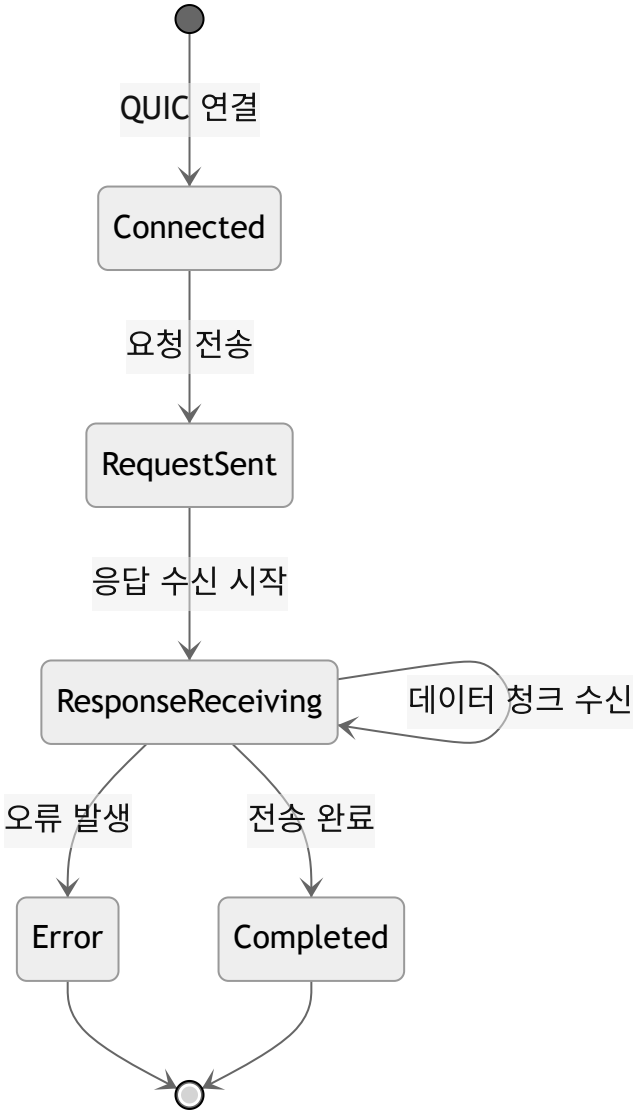
3.8 연결 및 스트림 관리

3.8.1 QUIC 연결 설정

```
// 클라이언트 측
let (writer, reader) = connection.open_bi().await?;


// 서버 측
while  let Ok((writer, reader)) = connection.accept_bi().await {
let  request_id = reader.id().index();
tokio::spawn(handle_stream(store, reader, writer));
}
```

3.8.2 스트림 생명주기



3.8.3 오류 처리

연결 종료 코드

```
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]

#[repr(u16)]

pub  enum Closed {

StreamDropped = 0, // 스트림 드롭

ProviderTerminating = 1, // 서버 종료

RequestReceived = 2, // 요청 수신 완료
```

```
}
```

오류 전파

```
impl Closed {
pub fn reason(&self) -> &'static [u8] {
match self {
Closed::StreamDropped => b"stream dropped",
Closed::ProviderTerminating => b"provider terminating",
Closed::RequestReceived => b"request received",
}
}
}
```

3.9 메시지 직렬화 및 파싱

3.9.1 요청 파싱

```
impl Request {
pub async fn read_async(
reader: &mut CountingReader<&mut RecvStream>,
) -> io::Result<Self> {
// 1. 요청 타입 읽기 (1바이트)
let request_type = reader.read_u8().await?;
```

```
let  request_type: RequestType = postcard::from_bytes(&[request_type])?;

// 2. 해당 구조체 역직렬화

Ok(match  request_type {

RequestType::Get => reader

.read_to_end_as::<GetRequest>(MAX_MESSAGE_SIZE)

.await?.into(),

RequestType::GetMany => reader

.read_to_end_as::<GetManyRequest>(MAX_MESSAGE_SIZE)

.await?.into(),

// ... 기타 타입들

})

}

}
```

3.9.2 크기 제한 및 보안

```
// 최대 메시지 크기 검증

if  request_bytes.len() > MAX_MESSAGE_SIZE {

return Err(RequestTooBigSnafu.build());

}


// 스트림별 단일 요청 제한
```

```
writer.finish().context(ClosedSnafu)?;
```

3.10 프로토콜 확장성

3.10.1 버전 관리

현재 ALPN `/iroh-bytes/4` 에서 숫자는 프로토콜 버전을 나타냅니다. 향후 호환성을 위해 슬롯 기반 확장을 지원합니다:

```
pub enum Request {
Get(GetRequest),
Observe(ObserveRequest),
Slot2, // 미래 확장용
Slot3, // 미래 확장용
// ...
}
```

3.10.2 하위 호환성

새로운 요청 타입이 추가되어도 기존 클라이언트는 알려진 타입만 처리하고, 알 수 없는 타입은 오류로 처리합니다.

3.11 성능 최적화

3.11.1 배치 처리

여러 작은 요청을 하나의 GetManyRequest로 묶어 라운드트립을 줄입니다:

```
// 비효율적: 개별 요청
for hash in small_files {
let request = GetRequest::blob(hash);
// 각각 별도 스트림...
}

// 효율적: 배치 요청
let request = GetManyRequest::from_iter(small_files);
// 단일 스트림으로 모든 파일 수신
```

3.11.2 스트림 재사용

QUIC의 다중 스트림 기능을 활용하여 연결을 재사용합니다:

```
// 동일한 연결에서 여러 스트림 생성
let (writer1, reader1) = connection.open_bi().await?;
let (writer2, reader2) = connection.open_bi().await?;
// 병렬 처리 가능
```

3.11.3 압축 최적화

ChunkRangesSeq의 효율적인 인코딩으로 메타데이터 오버헤드를 최소화합니다:

- 연속된 범위: 시작과 끝만 저장
- 반복 패턴: 런 길이 인코딩

- 차분 인코딩: 절대값 대신 차이값 저장

이러한 프로토콜 레이어의 설계는 효율적이고 확장 가능한 데이터 전송을 가능하게 하며, QUIC의 장점을 최대한 활용합니다.